



ClickHouse

No GIL, Real Gains

Porting a C++ Python extension to free-threaded Python 3.14.

Auxten Wang · Technical Director @ ClickHouse · Creator of chDB

PyData PyCon Yerevan · 24 Jul 2026

~~GIL~~

↓

3.14t

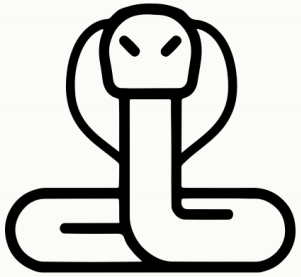
16 cores, one interpreter, zero subprocesses



Auxten Wang

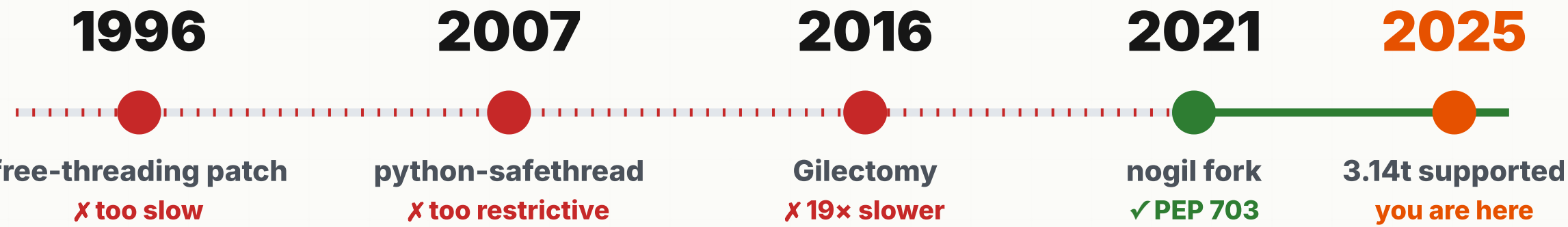
@auxten · auxten.com

- **Technical Director @ ClickHouse**
- **Creator of chDB — acquired by ClickHouse in 2024**
- **20+ years of C/C++ · systems & data infrastructure**
- **Ex-Shopee · CovenantSQL · Baidu · 360 · 4Paradigm**



ClickHouse

Killing the GIL: 30 years of attempts



Guido's bar (2007): remove the GIL only if single-threaded performance does not decrease.
Every attempt died on that bar — until one didn't. Here's each autopsy.

1996

Greg Stein's free-threading patch

One big lock → a small lock on *every* mutable structure. Python 1.4.

✕ ~2× slower single-threaded — shelved.

One survivor: `PyThreadState` **— still in CPython today.**

```
═══ comp.lang.python · 1996 ═══

From:    Greg Stein
Subject: free threading patches
         for Python 1.4

A patch set that removes the
global interpreter lock and
guards dicts, lists, modules…
with fine-grained locks.

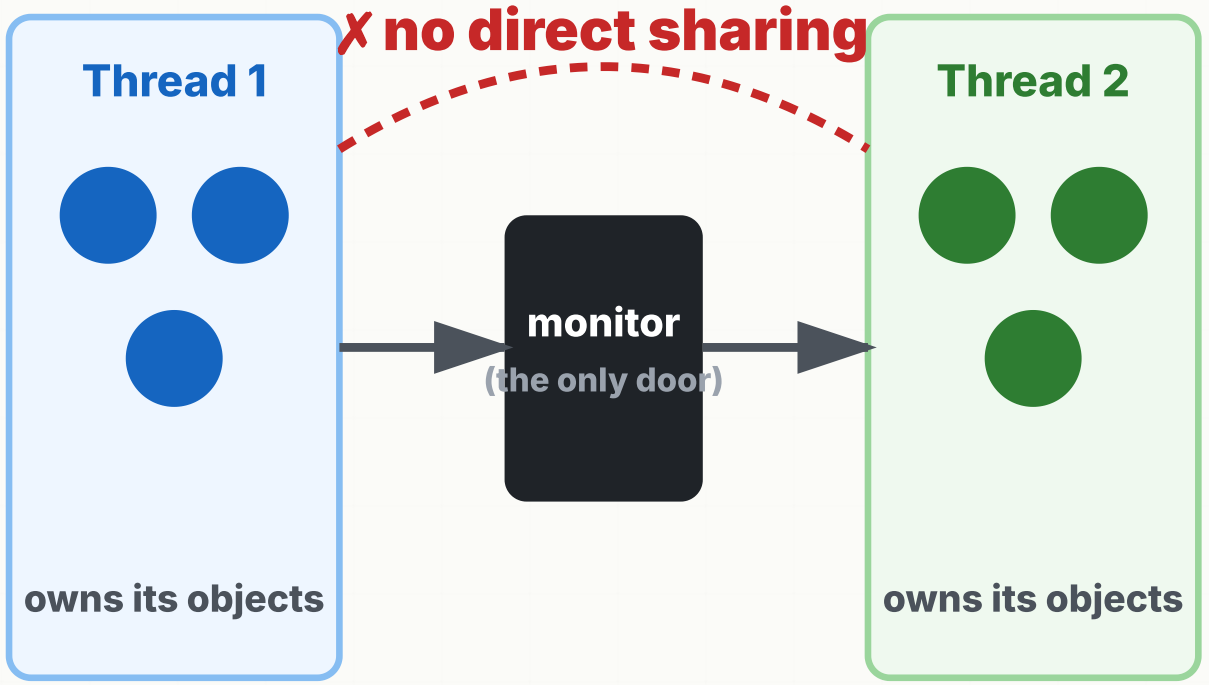
> benchmark: ~2x slowdown_
```

2007

Adam Olsen's python-safethread

Sidestep the locks: every object is *owned* by one thread. Sharing only through monitors.

✕ Safe — but you couldn't share anything freely. Faded away.



2016

Larry Hastings' Gilectomy

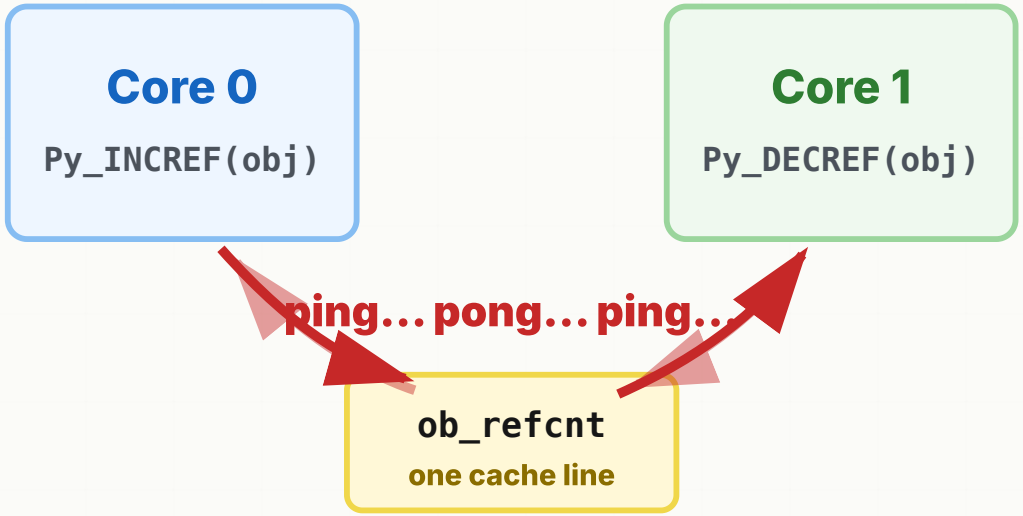
Make every refcount **atomic**. Now each `Py_INCREF` is a fight between CPU cores.

4.4 s → 83 s

18.9× slower on 7 cores. Abandoned.



Larry Hastings · PyCon 2016, Portland



every incref/decref = intercore bus traffic

2021

Sam Gross' nogil fork

Don't make refcounts atomic — redesign them so the common case never leaves the thread.

✓ Single-thread cost finally small → became PEP 703.



Sam Gross · author of the nogil fork & PEP 703

`PyObject — biased refcounting`

owner thread	everyone else
`count++ (plain)`	`atomic (rare)`

fast path stays on one core — no ping-pong

immortal objects: never counted at all

`None  True  42  ''  →  refcnt = ∞`

The one that landed: free-threaded Python

PEP 703 · 3.13

The GIL becomes optional. Experimental build, ~40% single-thread penalty.

PEP 779 · 3.14

Officially supported. Specializing interpreter enabled; penalty down to ~5–10%.

Opt-in build

A separate binary: `python3.14t`. Wheels get a `t` suffix: `cp314t`.

```
$ python3.14t -c "import sys; print(sys._is_gil_enabled())"
False        # threads now run Python bytecode in parallel
```

Phase III — making it *the default* Python — is on the roadmap. This train is leaving.

Why it matters: Python finally spans the cores

Single-core speed plateaued years ago — machines got *wider* instead. Free-threading lets one Python process use the width.

Agent builders

Parallel subagents, one process — shared models & caches. No pickle, no IPC.

Desktop apps

Responsive UI + truly parallel background work.

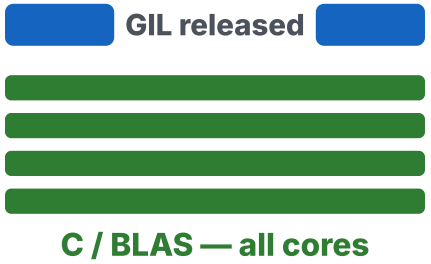
Servers & data tools

One copy of the model — not 16 workers × 16 copies.

Same language, same process, 16× the compute. The rest of this talk: what that costs extension authors.

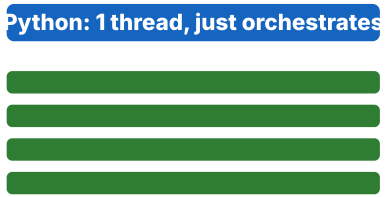
20 years of dodging the GIL: the four escape hatches

Drop to C



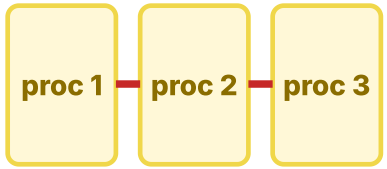
NumPy & SciPy: `Py_BEGIN_ALLOW_THREADS` around every kernel.

Thread pool below Python



TensorFlow & PyTorch: C++ thread pools, CUDA streams — the graph runs *outside* Python.

Fork & pickle



copies cross — the pickle tax

`multiprocessing`, joblib, DataLoader workers — every value crosses by **copy**.

Compile it away

```
with nogil:       # Cython
    crunch(buf)

@njit(nogil=True)  # Numba
def crunch(buf): ...
```

Leave the interpreter entirely — then threads are free.

Four hatches, one rule: never touch Python objects from a thread.

It works — until the work *is* Python

Python — one thread, holds the GIL, orchestrates

df.groupby(...) model(x) query(sql)

the GIL toll gate

objects cross one at a time

C / C++ / CUDA — 16+ threads, embarrassingly parallel

BLAS kernels · Eigen & ATen pools · ClickHouse pipelines · OpenMP

Python callback / UDF
→ back through the gate

object-dtype column
→ every cell is a PyObject

strings
→ PyUnicode, one by one

per-batch glue code
→ serial again

The moment threads need Python objects, every library hits the same wall. That wall is what 3.14t removes.

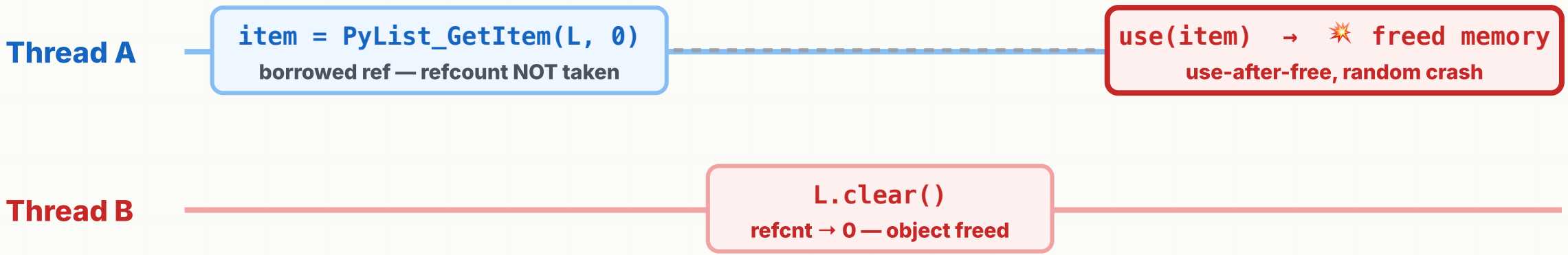
Extensions get nothing for free

- ✕ **One bad import poisons the well. Any extension without the opt-in flag → CPython silently *turns the GIL back on* for everyone.**
- ✕ **"Thread-safe because GIL" is now a data race. Every global, cache and lazy init loses its silent protection.**
- ✕ **Your abi3 wheel won't load. No stable ABI on 3.14t — new wheels, new CI.**

```
$ python3.14t -c "import _chdb"
RuntimeWarning: The global interpreter lock (GIL) has been enabled to load
module '_chdb', which has not declared that it can run safely without the GIL.
```

Free-threading is opt-in, per extension module. This talk: what opting in actually took.

Touching an object used to be free. Not anymore.



With the GIL, this interleaving was impossible — B couldn't run while A held the lock.
On 3.14t it's Tuesday.

`PyList_GetItem · borrowed` → `PyList_GetItemRef · strong` — whole C-API families had to change

The black magic that keeps 3.14t fast

Biased refcounting

owner
++

others
atomic

Two counters. The hot one never leaves its core.

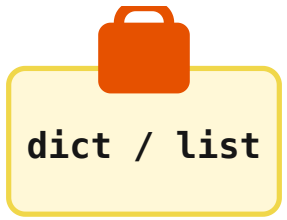
Immortal objects

`None True 42`

refcnt = ∞

The hottest objects opt out of counting entirely.

Per-object locks



Mutations lock *one object*, not the world — deadlock-proof critical sections.

Lock-free reads

`d[k]  d[k]  d[k]`

`no lock taken`

memory reclaimed only when readers are done (QSBR)

`dict`/`list` reads stay wait-free on the hot path.

This cocktail is why 3.14t costs ~5–10% where Gilectomy cost 19×.

Every big library re-did its object plumbing

WHO	WHAT THEY ACTUALLY DID
CPython C API	New primitives for everyone: `PyMutex`, critical sections, borrowed→strong `*_GetItemRef`
NumPy	Audited every C global & cache; all locks ported to `PyMutex`
PyTorch	Thread-safety audit of ATen & autograd → `cp314t` wheels since 2.10
TensorFlow	Hasn't — its parallelism lives below Python anyway, so the least pressure to move
Cython	One directive — `freethreading_compatible=True` — plus `cython.pymutex` for your hot spots
PyO3 (Rust)	Data races won't compile — `Send`/`Sync` checked by the compiler
chDB	…that's the rest of this talk

None of this is app code. It's plumbing, done once per library — so the other 99% of Python users never see it.

So who has opted in? (July 2026)

61%

of the top 360 compiled packages on PyPI now ship free-threaded wheels

219 / 360 · hugovk.dev/free-threaded-wheels · 15 Jul 2026

The scientific stack led the way. Chances are your daily imports already run free-threaded:

- NumPy
- SciPy
- pandas
- scikit-learn
- PyArrow
- Matplotlib
- Pillow
- PyTorch
- numba
- SQLAlchemy
- lxml
- aiohttp
- uvloop
- pydantic-core
- cryptography
- tiktoken
- psutil
- regex

The tooling has converged too: manylinux dropped 3.13t in May 2026 — 3.14t is *the* free-threaded target now.

...and who hasn't (yet)

PACKAGE	FT WHEELS	STATUS
NumPy	✓	First mover — wheels since 2.1 (Aug 2024)
pandas	✓	`cp314t` since 2.3.3 — Windows FT wheels still missing
PyTorch	✓	Preview in 2.9 → real `cp314t` wheels in 2.10 (Jan 2026)
chDB	✓	That's this talk
Polars	✗	Importing it re-enables the GIL for your whole process
grpcio · protobuf	✗	Blocks every gRPC-dependent tree (much of the AI SDK stack)
TensorFlow · DuckDB · tokenizers · orjson	✗	Not yet — watch the tracker

One ✗ anywhere in your dependency tree and the whole process quietly runs with the GIL back on. Trust nothing — check `sys._is_gil_enabled()` at runtime.

Our patient — chDB: a rocket engine on a bicycle

The ClickHouse engine, embedded in your Python process — SQLite-simple, built to scan millions of rows.

```
# pip install chdb
import chdb
chdb.query("SELECT count() FROM file('hits.parquet')")
```

- **In-process:** no server, no ports — just a module.
- **Same engine:** ClickHouse SQL, JOINs, vectors.
- **Multi-threaded C++ inside** — remember this bit.



Local like SQLite. Analytical like ClickHouse.

It drops into your Python data stack

70+ formats

Parquet, CSV, JSON, Arrow, ORC — read & write, no converters.

Zero-copy Pandas / Arrow

Query a live DataFrame in place: `SELECT … FROM Python(df)`

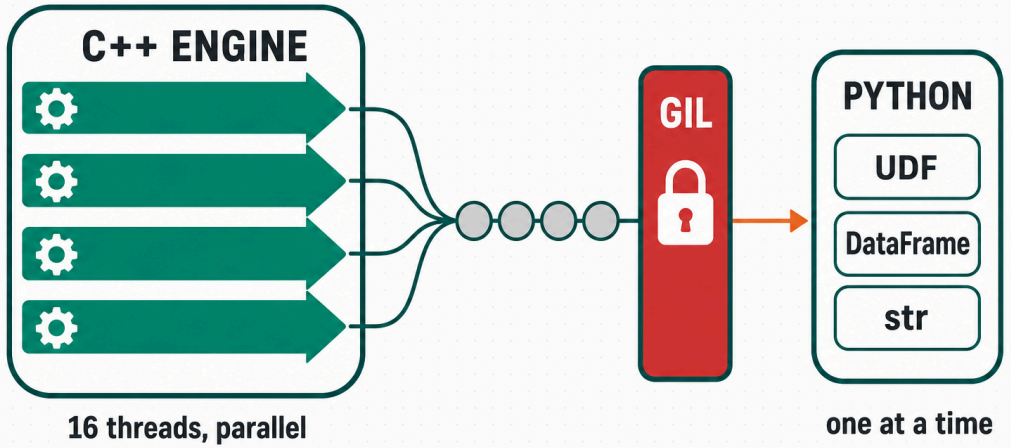
Python UDFs

Your Python functions callable from SQL — keep this one in mind too.

No ClickHouse knowledge needed today. For this talk, chDB is just "a multi-threaded C++ engine behind pybind11, reading Python objects."

Parallel engine × serial glue = serial query

The C++ engine releases the GIL and scales fine — but every stream queues on it to touch anything Python.



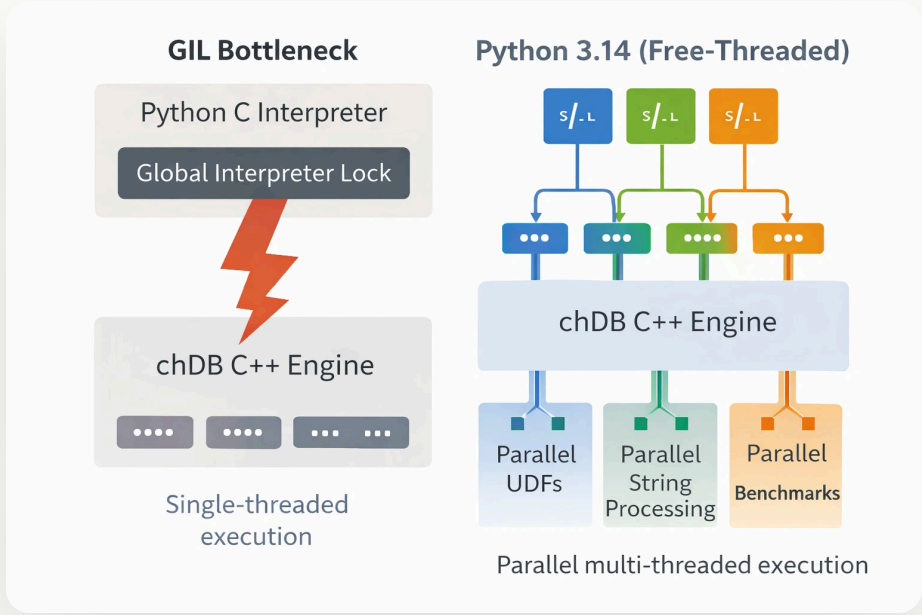
The GIL didn't slow our engine down. It serialized every conversation with Python.

Three changes that made no-GIL pay off

01 **Move Python UDF execution in-process**
Kill the subprocess pipeline first.

02 **Declare the module GIL-independent**
`Py_mod_gil_not_used` — then survive the audit it implies.

03 **Drop to low-level CPython string access**
On hot paths, the friendly Unicode APIs had to go.



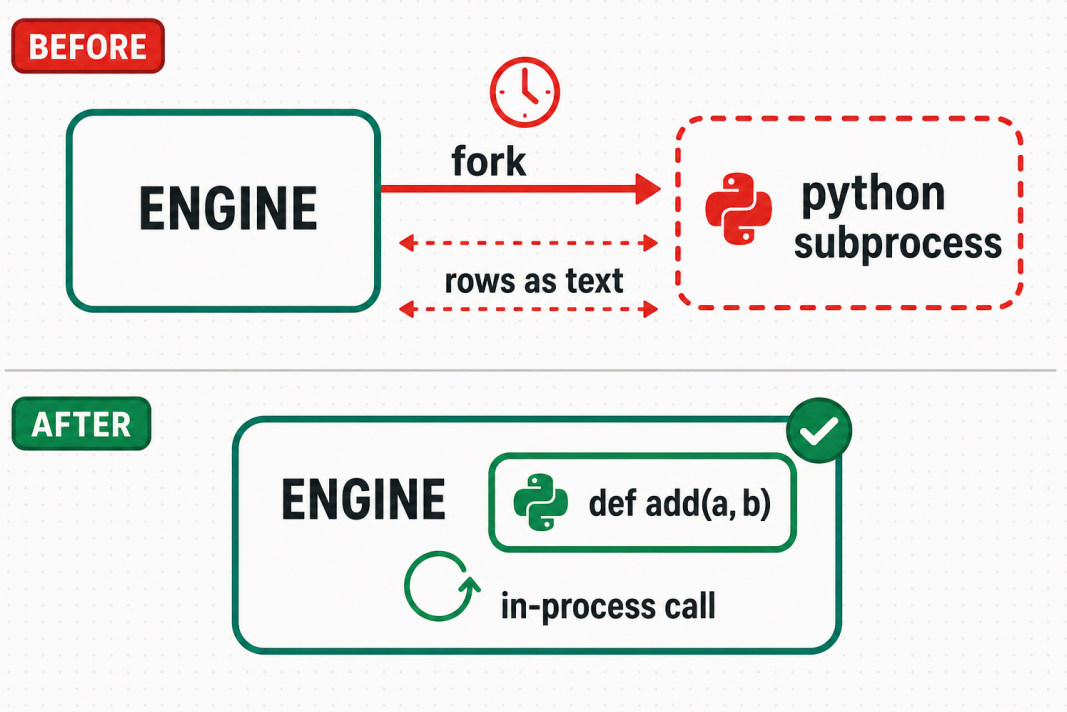
Plus the honest part: **what broke**, what didn't scale, and where the stable ABI stopped being an option.

UDFs ran out of process

The old decorator wrote a script + XML config; the engine forked Python and piped rows through it as text.

- ✕ Fork + interpreter startup on the query path
- ✕ Rows serialized as TabSeparated text over stdin/stdout
- ✕ Every argument arrives as `str` — no types
- ✕ No shared state with your process; imports re-run

The GIL wasn't even the bottleneck yet — the `fork()` was.



After: native in-process UDFs

C++ holds your py::function and calls it from the engine pipeline. Typed args, zero subprocesses.

```python
import chdb
from chdb.sqltypes import INT64

@chdb.func([INT64, INT64], INT64)  # in-process
def add(a, b):
    return a + b

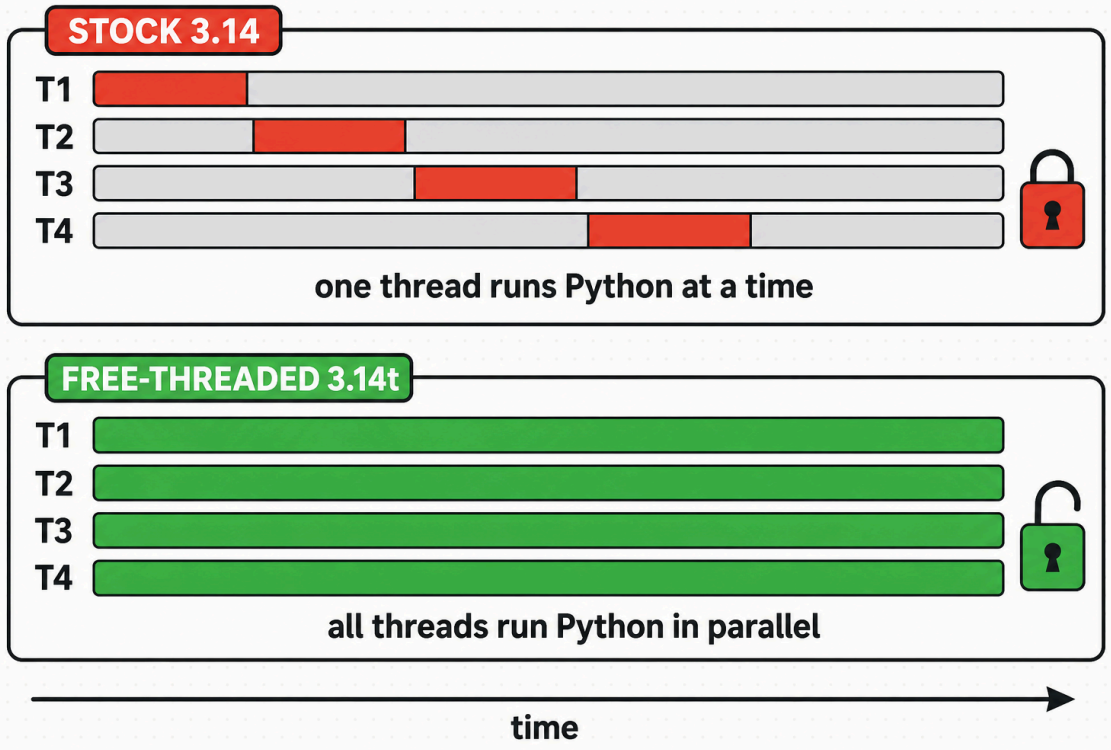
chdb.query("SELECT add(x, y) FROM t")
```

```cpp
// C++: PythonScalarUDF::executeImpl
// (called from engine worker threads)
py::gil_scoped_acquire acquire;   // <-- THE lock

for (size_t row = 0; row < rows; ++row)
{
    py::tuple py_args = box_arguments(row);
    py::object out    = func(*py_args); // your fn
    insert_into_column(result, out);
}
```

Faster already — but see that `gil_scoped_acquire`? On a stock build, every engine thread queues on it.

This is what no-GIL was made for



- **Stock 3.14: 16 streams hit the UDF → 16 threads take turns on one GIL. Effectively single-core.**
- **3.14t: the same `acquire` becomes per-thread bookkeeping — streams run your function in parallel.**
- **Your UDF must be thread-safe too — it now runs concurrently.**
- **Per-row boxing still costs: no-GIL removes the contention, not the call overhead.**

Order matters: go in-process *first*, then remove the GIL. A subprocess pipeline gains nothing from free-threading.

Opting in: one line — and an audit

```
PYBIND11_MODULE(_chdb, m, py::mod_gil_not_used())   // that's the whole declaration
```

The line is easy. It's a **claim about every global in your codebase** — so the real work was a state audit:

- **Import cache, UDF registry, engine singleton — every "thread-safe because GIL" spot got a real lock (`call_once`, `shared_mutex`).**
- **Textbook C++ concurrency — the hard part is finding it, not fixing it.**
- **Our import cache's "safe" lock-free fast path had been a data race for years. The GIL was its alibi.**

None of the fixes were exotic. The GIL was just hiding the to-do list.

The nastiest bug: lazy init on a GIL-free thread

Trigger

First `NULL` in a string column → a scan thread lazily imports `pandas.NaT`... with no Python thread state.

Crash

Segfault deep inside the allocator — nowhere near the actual bug.

Hang

Then the fatal-signal handler deadlocks. No traceback, no exit. The query just hangs, forever.

The fix was two lines: resolve everything eagerly, while you still hold the GIL. Finding it took days — debugging *silence* is the expensive part.

Strings are the hot path — go straight to the bytes

Analytics = millions of strings crossing the C++/Python border per query. The friendly Unicode APIs cost a function call *per value* — so on 3.14t we read CPython's memory directly:

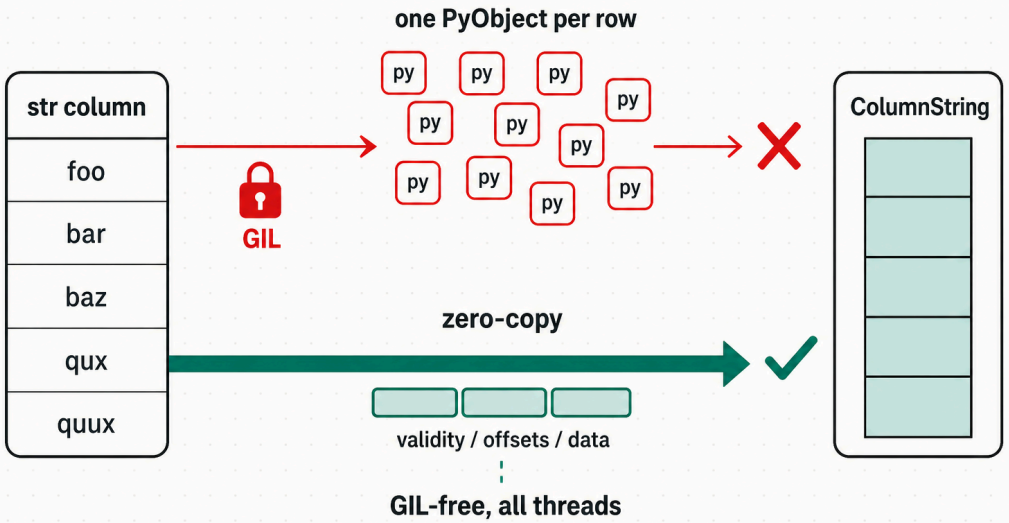
```
if (PyUnicode_IS_COMPACT_ASCII(obj)) {                    // most analytical data
    data = (const char *) PyUnicode_1BYTE_DATA(obj);      // direct pointer, no copy
    len  = PyUnicode_GET_LENGTH(obj);
}
// ...cached-UTF8 / compact / legacy fallbacks omitted
```

- **ASCII string → a pointer read. No shim, no allocation, safe from any thread.**
- **Free-threaded builds can't use the stable ABI anyway — nothing left to lose by going low-level.**

Rule of thumb on hot paths: the friendlier the API, the more it costs per row.

The biggest win: skip PyObject entirely

pandas 3.x `str` columns are **Arrow-backed**: plain C buffers, scanned directly on all streams — *no GIL at all*.



object dtype (before) — 219.2 s

Arrow zero-copy (after) — 20.1 s

ClickBench, 50M-row hits DataFrame, all queries, pandas 3.x, hot run · **10.9×** total · geomean **4.5×** · string-heavy up to **64×**

Priority order on hot paths: no PyObject > cheap PyObject access > friendly PyObject APIs.

What broke, what surprised us

- ✕ **The silent GIL re-enable got us too. One undeclared dependency and our "no-GIL benchmark" was measuring the GIL.**
- ✕ **Crashes hide as hangs. Allocator segfault → signal-handler deadlock. Budget time for debugging *silence*.**
- ✕ **New wheels, new CI. No stable ABI on 3.14t → separate `cp314t` build lanes, doubled CI matrix.**
- ✕ **No-GIL didn't speed up what was already parallel. Only the Python touchpoints improved — and single-threaded got ~5–10% slower.**

Free-threading didn't *create* these bugs. It revoked the amnesty they'd been living under.

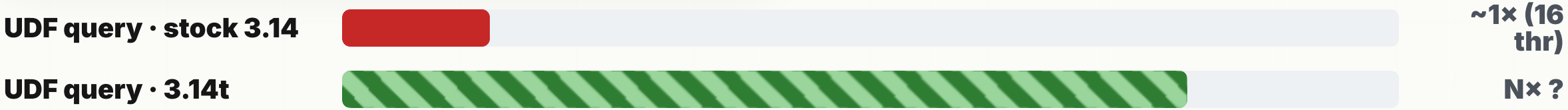
Numbers from a 16-core machine DRAFT

Shipped & measured

- ✓ ClickBench, 50M rows: 219.2s → 20.1s (10.9×)
- ✓ String-heavy queries: up to 64×
- ✓ CI guard: 5M-string ingestion microbench on 3.14

To re-run for this talk TODO

- UDF-heavy query: threads 1→16, stock vs 3.14t
- Object-dtype string scan: stock vs 3.14t
- Single-thread regression: 3.14 vs 3.14t



Striped bar = placeholder — final scaling curve to be measured on the 16-core box before the talk.

When does no-GIL actually help?

It pays when…

- ✓ Many threads must execute Python (UDFs, callbacks)
- ✓ Many threads must read PyObjects (object-dtype columns)
- ✓ The GIL was your *measured* bottleneck, not a hunch

It doesn't when…

- ✗ Your C++ already releases the GIL for the heavy work
- ✗ You can avoid Python objects entirely (Arrow, buffers, NumPy)
- ✗ The workload is single-threaded — you pay ~5–10% for nothing

Best result in our port: zero-copy beat lock-removal. Free-threading fixes what zero-copy can't reach — running actual Python in parallel.

Own an extension? The 60-second version

For the few of you maintaining a C/C++/Rust extension — photograph this slide:

1. **Run your tests on `3.14t` as-is. Watch for the GIL re-enable warning.**
2. **Kill subprocesses and single workers first — give threads something to win.**
3. **Globals, caches, singletons → real locks. The GIL was their alibi.**
4. **Pre-resolve lazy imports reachable from non-Python threads.**
5. **Declare `Py_mod_gil_not_used` *last* — then benchmark scaling and the single-thread regression.**

Everyone else: you get all of this for free the moment your dependencies opt in. That's what the 61% wall is buying you.

Three takeaways

01 **No-GIL is opt-in, and the opt-in is an audit.**
The declaration is one line; the promise covers every global you own.

02 **Restructure before you parallelize.**
In-process beats out-of-process; zero-copy beats lock-free; then free-threading multiplies what's left.

03 **The GIL was an amnesty, not a feature.**
3.14t revokes it. The bugs it exposes were already yours.

Try it on your own extension

```
pip install chdb           # try the engine
python3.14t your_tests.py # try the future
```

Code & slides

`github.com/chdb-io/chdb`
`auxten.com · @auxten`

Read more

py-free-threading.github.io
PEP 703 · PEP 779 · chDB docs:
clickhouse.com/docs/chdb

Python can finally scale across cores.
Now it's our extensions' turn.





Q&A

`github.com/chdb-io/chdb` · `auxten.com`

No GIL, real gains — thank you, Yerevan!